National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 2: MLP Classifier Report

Submitted to:
Dr. Shahbaz Khan

Submitted by:
Hamza Suhaib Qarni
538866
SMME

April 10, 2026

Contents

1 Introduction **2**

2 Code Implementation: Original vs. Modified **2**
2.1 Original Code (Static) 2
2.2 Improved Code (Dynamic Exploration for Tasks 1 & 2) 2

3 Task 1: Hidden Layer Configuration Search **3**
3.1 Observations 3

4 Task 2: Learning Rate Search **4**
4.1 Observations 4

5 Conclusion **5**

1 Introduction

This report outlines the implementation and evaluation of a Multi-Layer Perceptron (MLP) for the Iris classification dataset (Setosa, Versicolor, Virginica). The primary objective of this tutorial is to observe the empirical effects of altering the neural network's architecture (hidden layers/neurons) and the learning rate.

Data preprocessing included a 70/30 train-test split and standard scaling (`StandardScaler`) to ensure optimal gradient descent convergence.

2 Code Implementation: Original vs. Modified

To address the tasks, the original static MLP implementation was modified to iterate through multiple configurations programmatically.

2.1 Original Code (Static)

In the base tutorial, the model was initialized with a single fixed configuration:

```
from sklearn.neural_network import MLPClassifier

# Original static initialization
mlp = MLPClassifier(hidden_layer_sizes=(10,),
    max_iter=1000, random_state=42)
mlp.fit(X_train_scaled, y_train)
```

2.2 Improved Code (Dynamic Exploration for Tasks 1 & 2)

To systematically test architectures and learning rates, loops were introduced to track performance metrics dynamically:

```
# Task 1: Iterating over varying Hidden Layer
    Configurations
configs = [(1,), (2,), (3,), (5,), (10,), (20,),
    (50,), (100,),
(10, 10), (20, 20), (50, 50), (100, 100),
(50, 30, 10), (100, 50, 25)]

for config in configs:
mlp = MLPClassifier(hidden_layer_sizes=config,
    max_iter=1000,
random_state=42)
# Fit model and record accuracy, loss curve, and
    epochs...

# Task 2: Iterating over varying Learning Rates (
    using best config (5,))
learning_rates = [0.0001, 0.001, 0.005, 0.01,
    0.05, 0.1, 0.5]

for lr in learning_rates:
```

```
                    mlp = MLPClassifier(hidden_layer_sizes=(5,),
                        learning_rate_init=lr,
                    max_iter=1000, random_state=42)
                    # Fit model and record metrics...
```

3 Task 1: Hidden Layer Configuration Search

The network was tested with shallow (single-layer) and deep (multi-layer) configurations to evaluate capacity vs. performance.

3.1 Observations

- **Underfitting:** A single neuron `(1,)` lacks the capacity to map the dataset, capping at 0.5556 accuracy.
- **Optimal Capacity:** The `(5,)` configuration is the most efficient, achieving perfect 1.0000 accuracy without unnecessary overhead.
- **Diminishing Returns:** Deep architectures like `(100, 50, 25)` increase training time significantly (up to 0.625s) without providing any additional accuracy benefit for this simple dataset.

Table 1: Performance Comparison of Hidden Layer Configurations

Configuration	Accuracy	Epochs	Time (s)
(1,)	0.5556	1000	0.388
(3,)	0.9556	1000	0.407
(5,)	**1.0000**	**1000**	**0.407**
(50,)	1.0000	591	0.315
(100, 100)	1.0000	400	0.714
(100, 50, 25)	1.0000	357	0.625

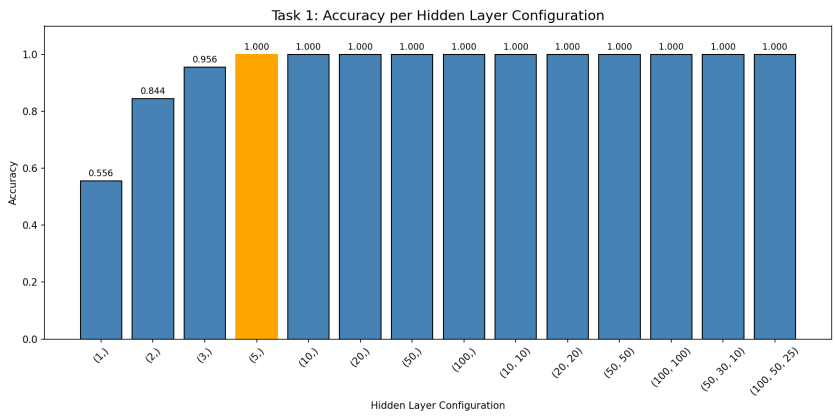

Figure 1: Accuracy per configuration. `(5,)` is the threshold for perfect accuracy.

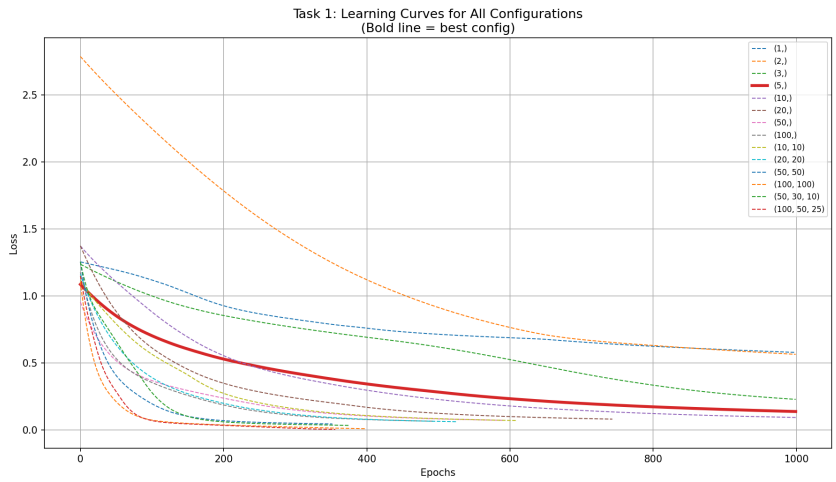

Figure 2: Learning Curves. The bold line indicates the `(5,)` configuration.

4 Task 2: Learning Rate Search

Fixing the architecture to `(5,)`, the initial learning rate was varied from 0.0001 to 0.5 to find the optimal step size.

4.1 Observations

- **Too Slow:** LR = 0.0001 failed to converge within 1000 epochs (0.7111 accuracy).
- **Optimal Balance:** LR = 0.5 drastically reduced convergence time, achieving 1.0000 accuracy in only 64 epochs. For this standard scaled dataset, a high learning rate does not overshoot the minimum.

Table 2: Learning Rate Performance (Fixed Config: (5,))

Learning Rate	Accuracy	Epochs	Time (s)	Balance Score
0.0001	0.7111	1000	0.425	0.000
0.001	1.0000	1000	0.465	0.500
0.01	1.0000	353	0.172	0.846
0.1	1.0000	76	0.040	0.994
0.5	**1.0000**	**64**	**0.032**	**1.000**

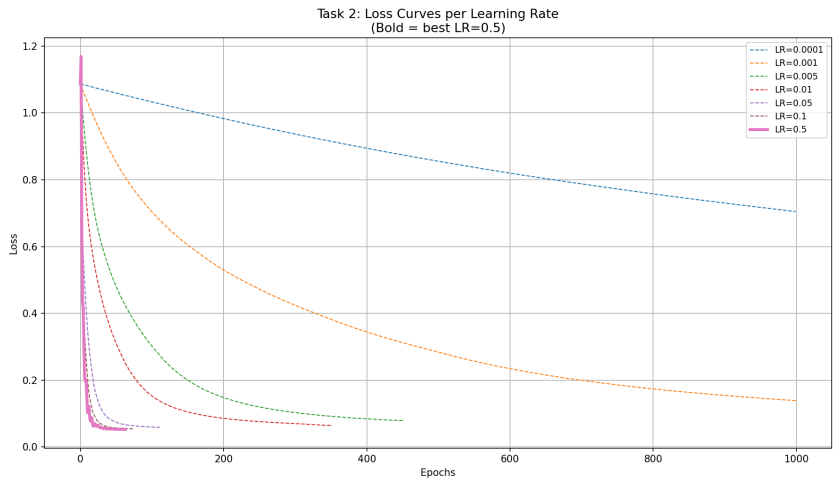

Figure 3: Loss Curves for varying Learning Rates. LR=0.5 yields the steepest descent.

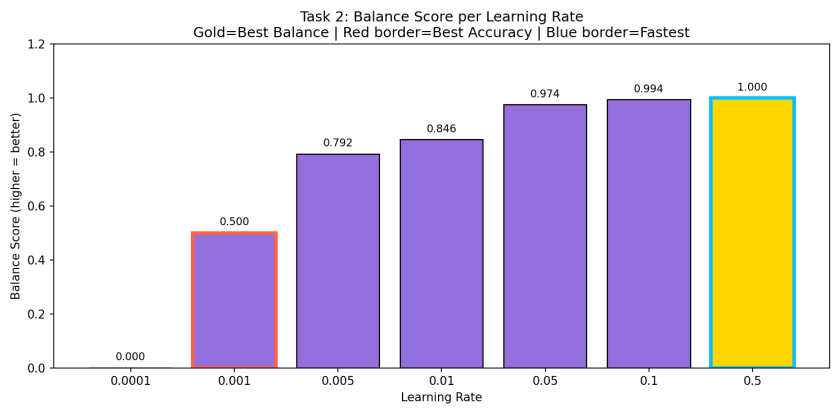

Figure 4: Balance Score identifying LR=0.5 as the optimal trade-off point.

5 Conclusion

The final tuned MLP model uses a single hidden layer of 5 neurons with an initial learning rate of 0.5. It successfully classifies the dataset with 1.0000 accuracy, precision, and recall in just 64 epochs. The results demonstrate that appropriately matching network capacity and learning rate to dataset complexity prevents unnecessary computational waste and speeds up training.